Tendermint

📋 개요

Tendermint는 2016년 Jae Kwon과 Ethan Buchman이 개발한 **블록체인 지향 BFT 합의 알고리즘**입니다. Cosmos 네트워크의 핵심이며, PBFT를 개선하여 블록체인 환경에 최적화했습니다.

핵심 특징

- **Round-based Voting**: 라운드 기반 투표 시스템
- **Lock 메커니즘**: 안전성을 위한 잠금 구조
- **Instant Finality**: 블록 확정 즉시 최종성
- **2f+1 Quorum**: n = 3f + 1 노드 구조
- **O(n^2) 메시지 복잡도**: 모든 노드가 투표

🎯 핵심 개념

라운드 구조

```
Round = {
    Proposer: 리더 (round-robin)
    Prevote:  첫 번째 투표
    Precommit: 두 번째 투표
    Commit:   최종 커밋
}
```

Lock 메커니즘

- **LockedValue**: 노드가 잠근 블록
- **LockedRound**: 잠금 발생한 라운드
- **ValidValue**: 마지막으로 유효했던 블록
- **ValidRound**: 유효 블록의 라운드

Quorum

```
n = 4, f = 1
Quorum = 2f + 1 = 3

Prevote quorum:  3 votes
Precommit quorum: 3 votes
```

🔄 동작 과정

전체 흐름도

```
Proposer            Validator1          Validator2          Validator3
  |                     |                   |                   |
  |---PROPOSAL--------->|                   |                   |
  |---PROPOSAL--------->|------------------>|                   |
  |---PROPOSAL--------->|------------------>|------------------>|
  |                     |                   |                   |
  |                     |                   |                   |
  |     [Prevote Round - All validators vote]                     |
  |                     |                   |                   |
  |<---PREVOTE----------|                   |                   |
  |<---PREVOTE----------|<------------------|                   |
  |<---PREVOTE----------|<------------------|<------------------|
  |----PREVOTE--------->|                   |                   |
  |----PREVOTE--------->|------------------>|                   |
  |----PREVOTE--------->|------------------>|------------------>|
  |                     |                   |                   |
  |     [Precommit Round - All validators precommit]              |
  |                     |                   |                   |
  |<---PRECOMMIT--------|                   |                   |
  |<---PRECOMMIT--------|<------------------|                   |
  |<---PRECOMMIT--------|<------------------|<------------------|
  |----PRECOMMIT------->|                   |                   |
  |----PRECOMMIT------->|------------------>|                   |
  |----PRECOMMIT------->|------------------>|------------------>|
  |                     |                   |                   |
  |     [Commit - Execute block]                                  |
```

Round 구조 상세

```
┌──────────────────────────────────────────────┐
│              Round R                         │
├──────────────────────────────────────────────┤
│ 1. Propose Phase                             │
│    - Proposer broadcasts PROPOSAL            │
│    - Contains block + PoL (if exists)        │
├──────────────────────────────────────────────┤
│ 2. Prevote Phase                             │
│    - All validators vote on proposal         │
│    - Wait for 2f+1 prevotes                  │
│    - Can prevote NIL                         │
├──────────────────────────────────────────────┤
│ 3. Precommit Phase                           │
│    - Validators precommit if conditions met  │
│    - Wait for 2f+1 precommits                │
│    - Can precommit NIL                       │
├──────────────────────────────────────────────┤
│ 4. Commit                                    │
│    - If 2f+1 precommits same block           │
│    - Execute and move to next height         │
│    - Otherwise, go to Round R+1              │
└──────────────────────────────────────────────┘
```

Phase 1: Propose

Proposer 선출:

```
proposer = (height + round) % numValidators
```

동작:

1. Proposer가 새 블록 생성
2. PoL (Proof of Lock) 포함 (있다면)
3. 모든 Validator에게 PROPOSAL 전송

메시지 구조:

```
PROPOSAL {
    height:   현재 블록 높이
    round:    현재 라운드
    block:    제안 블록
    PoL:      Proof of Lock (optional)
```

```
    proposer: 제안자 ID
}
```

PoL (Proof of Lock):

- 이전 라운드에서 2f+1 prevote 받은 증거
- 새 proposer가 이전 진행 상황 계속

Phase 2: Prevote

목적: 제안된 블록에 대한 첫 번째 투표

Prevote 규칙:

```
if validBlock && (notLocked || lockedBlock == proposedBlock) {
    prevote(proposedBlock)
} else if validBlock && lockedRound < proposedPoLRound {
    // Unlock and prevote new block
    unlock()
    prevote(proposedBlock)
} else {
    prevote(NIL)
}
```

케이스 분석:

1. **케이스 1: 블록 유효 + 잠금 없음**

```
✓ 유효한 블록
✓ 잠금 없음
→ Prevote(block)
```

2. **케이스 2: 블록 유효 + 같은 블록에 잠금**

```
✓ 유효한 블록
✓ 잠긴 블록과 동일
→ Prevote(block)
```

3. **케이스 3: 블록 유효 + 다른 블록에 잠금 + PoL 있음**

```
✓ 유효한 블록
✓ 다른 블록에 잠금
```

```
✓ PoL의 라운드 > 내 잠금 라운드
→ Unlock + Prevote(block)
```

4. **케이스 4: 그 외**

```
→ Prevote(NIL)
```

메시지 구조:

```
PREVOTE {
    height:    현재 블록 높이
    round:     현재 라운드
    blockHash: 투표하는 블록 (또는 NIL)
    validator: 투표자 ID
}
```

Phase 3: Precommit

목적: 최종 커밋 전 두 번째 투표 + Lock

Precommit 규칙:

```
if receivedPrevoteQuorum(block) {
    // Lock on this block
    lockedValue = block
    lockedRound = currentRound
    precommit(block)
} else if receivedPrevoteQuorum(NIL) {
    // Unlock
    unlock()
    precommit(NIL)
} else {
    // Timeout, no quorum
    precommit(NIL)
}
```

Lock 설정:

```
// 2f+1 prevotes for block B
lockedValue = B
lockedRound = R
```

```
validValue = B
validRound = R
```

Lock의 의미:

- 이 노드는 이제 B만 precommit
- 더 높은 라운드의 PoL이 없으면 unlock 안 함
- 안전성 보장의 핵심

메시지 구조:

```
PRECOMMIT {
    height:    현재 블록 높이
    round:     현재 라운드
    blockHash: 투표하는 블록 (또는 NIL)
    validator: 투표자 ID
}
```

Phase 4: Commit

커밋 조건:

```
if receivedPrecommitQuorum(block) {
    commit(block)
    moveToHeight(height + 1)
}
```

실행:

1. 블록을 블록체인에 추가
2. 상태 머신 업데이트
3. 다음 높이로 이동

📊 상태 머신

노드 상태

```
┌──────────────┐
│   Propose    │──┐
```

```
└──────────────┘  |
        |         |
        ▼         | timeout/
┌──────────────┐  | new round
|   Prevote    |  |
└──────────────┘  |
        |         |
        ▼         |
┌──────────────┐  |
|  Precommit   |  |
└──────────────┘  |
        |         |
        ▼         |
┌──────────────┐  |
|    Commit    |──┘
└──────────────┘
```

상태 전이 조건

```
// Propose → Prevote
if receivedProposal() || timeout() {
    moveToPrevote()
}

// Prevote → Precommit
if receivedPrevoteQuorum() || timeout() {
    moveToPrecommit()
}

// Precommit → Commit or NewRound
if receivedPrecommitQuorum(block) {
    commit()
    moveToNewHeight()
} else if receivedPrecommitQuorum(NIL) || timeout() {
    moveToNewRound()
}
```

🔒 Lock 메커니즘 상세

Lock이 필요한 이유

문제 시나리오:

```
Round 1:
- Validator A, B prevote Block X
- Validator C prevote Block Y
- No quorum

Round 2 (without lock):
- Validator A, B, C all prevote Block Y
- Block Y commits
- But Block X might have been committed by other nodes!
```

Lock으로 해결:

```
Round 1:
- Validator A, B, C prevote Block X
- All precommit Block X (locked!)
- Timeout before commit

Round 2:
- New proposer proposes Block Y
- But A, B, C are locked on X
- They prevote NIL (can't change)
- Y doesn't get quorum
```

Lock 예제 (4노드)

```
Height 1, Round 0:
Node 0: Proposer, proposes Block A

Prevote:
- Node 0, 1, 2: Prevote(A)  [3/4 = quorum!]
- Node 3: Offline

Precommit:
- Node 0, 1, 2: Precommit(A) + Lock(A, Round=0)
- Node 3: Still offline
- [3/4 = quorum... but timeout]

Height 1, Round 1:
Node 1: Proposer, proposes Block B

Prevote:
```

```
- Node 0, 1, 2: Still locked on A
- Node 0, 1, 2: Prevote(NIL)  [no PoL for B]
- No quorum for B

Height 1, Round 2:
Node 2: Proposer, proposes Block A (knows about lock)

Prevote:
- Node 0, 1, 2: Prevote(A)  [locked!]
- Node 3: Back online, Prevote(A)
- [4/4 quorum]

Precommit:
- All: Precommit(A)
- [4/4 quorum]

Commit:
- Block A committed!
```

📈 메시지 흐름 분석

4노드 예제 (n=4, f=1)

한 라운드의 메시지 수:

1. **Proposal**: Proposer → 3 validators = 3 messages
2. **Prevote**: 4 validators × 3 targets = 12 messages
3. **Precommit**: 4 validators × 3 targets = 12 messages

총: 3 + 12 + 12 = **27 messages per block**

배치 처리 (batch size = 10):

- 27 messages / 10 transactions = **2.7 messages/tx**
- 실제 구현: **4.8 messages/tx** (재전송, 타임아웃 포함)

타임라인

```
t=0:    Proposer broadcasts PROPOSAL
t=1:    All validators send PREVOTE (12 messages)
```

```
t=2:    Prevote quorum reached
t=3:    All validators send PRECOMMIT (12 messages)
t=4:    Precommit quorum reached
t=5:    Block committed, move to next height
```

🔍 안전성 및 활성

안전성 증명

정리: 두 정직한 validator가 같은 높이에서 다른 블록을 커밋하지 않음

핵심 논증:

1. 블록 커밋 = 2f+1 precommit 필요
2. 2f+1 precommit = 2f+1 prevote 필요 (같은 라운드 or PoL)
3. 2f+1 prevote = 최소 f+1 정직 노드
4. 정직 노드는 lock 메커니즘으로 보호
5. 두 블록이 모두 2f+1 precommit 받으려면 f+1 정직 노드가 두 블록에 투표해야 함
6. Lock 때문에 불가능!

활성 보장

조건:

- 동기 기간 (GST, Global Stabilization Time 이후)
- 정직한 proposer

보장:

- 정직한 proposer가 선출되면 블록 커밋
- Round-robin으로 결국 정직한 proposer 선출

💻 구현 세부사항

핵심 데이터 구조

```go
type TendermintNode struct {
    height int
    round  int
    step   Step  // Propose, Prevote, Precommit, Commit

    // Lock 관련
    lockedValue *Block
    lockedRound int
    validValue  *Block
    validRound  int

    // 투표 수집
    prevotes   map[string]map[int]bool  // [blockHash][validatorID]
    precommits map[string]map[int]bool  // [blockHash][validatorID]
}

type Step int
const (
    StepPropose Step = iota
    StepPrevote
    StepPrecommit
    StepCommit
)
```

주요 함수

```go
func (node *TendermintNode) propose() {
    if node.isProposer() {
        block := node.createBlock()
        node.broadcast(PROPOSAL, block)
    }
    node.schedulePrevote()
}

func (node *TendermintNode) prevote() {
    var blockHash string

    if node.validProposal() && node.canPrevote() {
        blockHash = node.proposedBlock.Hash()
    } else {
        blockHash = "NIL"
    }

    node.broadcast(PREVOTE, blockHash)
```

```go
    node.schedulePrecommit()
}

func (node *TendermintNode) precommit() {
    var blockHash string

    if node.receivedPrevoteQuorum() {
        blockHash = node.quorumBlock.Hash()
        node.lock(node.quorumBlock)  // Lock!
    } else {
        blockHash = "NIL"
        node.unlock()
    }

    node.broadcast(PRECOMMIT, blockHash)
    node.scheduleCommit()
}

func (node *TendermintNode) commit() {
    if node.receivedPrecommitQuorum() {
        node.executeBlock(node.quorumBlock)
        node.moveToNextHeight()
    } else {
        node.moveToNextRound()
    }
}
```

코드 위치

- 구현: `internal/algorithms/tendermint/tendermint.go`
- 테스트: `internal/algorithms/tendermint/tendermint_test.go`

🆚 PBFT vs Tendermint

특징	PBFT	Tendermint
Phase 수	3 (Pre-prepare, Prepare, Commit)	3 (Propose, Prevote, Precommit)
리더	Fixed Primary (view 변경 시만)	Round-robin
안전성 메커니즘	2f+1 quorum	2f+1 quorum + Lock

특징	PBFT	Tendermint
재시도	View change	Round 증가
블록체인	범용	블록체인 최적화
Finality	Eventually	Instant

Tendermint의 개선점

1. **Lock 메커니즘**:
    - PBFT: Implicit 순서 보장
    - Tendermint: Explicit lock으로 명확한 안전성
2. **Round 구조**:
    - PBFT: View change는 무거운 작업
    - Tendermint: Round 증가는 가벼움
3. **블록체인 친화적**:
    - Height 개념 내장
    - Instant finality

📈 성능 특성

장점

✅ Instant finality (블록 확정 즉시 최종)
✅ Lock 메커니즘으로 명확한 안전성
✅ Round-robin으로 공정한 리더 선출
✅ PBFT보다 간단한 재시도

단점

❌ $O(n^2)$ 메시지 복잡도
❌ 동기 가정 필요 (활성)

❌ 3단계 프로토콜

최적화

- **Batching**: 여러 트랜잭션을 하나의 블록으로
- **Pipelining**: 다음 블록 제안을 미리 준비
- **Compact block**: 블록 해시만 전송

🔬 벤치마크 결과

우리 구현 (4노드, 100트랜잭션):

- **TPS**: 1.0
- **메시지/트랜잭션**: 4.8
- **지연시간**: ~10ms (시뮬레이션)

PBFT와 비교:

- 메시지 수: Tendermint (4.8) < PBFT (8.1) ✓
- Lock 메커니즘으로 더 명확한 안전성 ✓

🎓 학습 포인트

1. **Lock의 중요성**:
    - 안전성 보장의 핵심
    - 한번 prevote quorum 받으면 lock
    - 더 높은 PoL 없으면 unlock 안 함
2. **Prevote vs Precommit**:
    - Prevote: “이 블록 괜찮아 보임”
    - Precommit: “이 블록에 커밋할 준비됨”
    - 2단계 투표로 안전성 강화
3. **Round 진행**:
    - Timeout이나 NIL quorum으로 다음 round

    - Round-robin으로 공정성
    - Lock으로 일관성 유지

🌟 실제 사용 사례

- **Cosmos Hub**: IBC 프로토콜
- **Binance Chain**: 거래소 체인
- **Terra**: 스테이블코인
- **Crypto.org Chain**: CRO 토큰

📚 참고 자료

- [원본 논문] Buchman, E. (2016). "Tendermint: Byzantine Fault Tolerance in the Age of Blockchains"
- [The latest gossip on BFT consensus] Buchman et al. (2018)
- [Cosmos 문서] https://docs.cosmos.network/
- [코드 구현] `internal/algorithms/tendermint/tendermint.go`

← PBFT | HotStuff Family →